

Q7. Hospital Emergency System – Thread Priority

CODE

```
class Main{
    public static void main(String[] args){
        HospTask emergency = new HospTask("Emergency Patient Processing");
        HospTask general = new HospTask("General Patient Processing");
        HospTask billing = new HospTask("Billing Processing");
        emergency.setPriority(Thread.MAX_PRIORITY);
        general.setPriority(Thread.NORM_PRIORITY);
        billing.setPriority(Thread.MIN_PRIORITY);

        try{
            emergency.start();
            emergency.join();

            general.start();
            general.join();

            billing.start();
            billing.join();
        }catch(InterruptedException e){
            System.out.println("Thread executionn interrupted: " + e.getMessage());
        }
    }
}

class HospTask extends Thread{
    private String taskName;
    public HospTask(String taskName){
        this.taskName = taskName;
    }
    public void run(){
        System.out.println(taskName+ "-Priority: " + getPriority() + "is running");
    }
}
```

OUTPUT

```
Emergency Patient Processing-Priority: 10is running
General Patient Processing-Priority: 5is running
Billing Processing-Priority: 1is running
```

Q8. Manufacturing Plant – Producer-Consumer Problem

— Not Submitted —

Q9. Online Banking – Exception Handling with Multiple Threads

— Not Submitted —

Student 23 **MALLI CHANDANA** 192472250RD

5 / 9 submitted

Q1. Banking Transaction System – Exception Handling

CODE

```
import java.util.*;
public class Main{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        try{
            int balance = sc.nextInt();
            int amount = sc.nextInt();
            if(amount<=0){
                throw new IllegalArgumentException("Withdrawal amount must be greater than zero.");
            }
            if(amount>balance){
                throw new InsufficientBalanceException("Insufficient balance for withdrawal.");
            }
            balance=balance-amount;
            System.out.println("withdrawal successfull.");
            System.out.println("Remaining Balance = "+balance);
        }
        catch(InsufficientBalanceException e){
            System.out.println(e.getMessage());
        }
        catch(IllegalArgumentException e){
            System.out.println(e.getMessage());
        }
        catch(Exception e){
            System.out.println("Invalid input.");
        }
        finally{
            System.out.println("Transaction completed.");
        }
    }
}
class InsufficientBalanceException extends Exception{
    InsufficientBalanceException(String message){
        super(message);
    }
}
```

INPUT

abc 3000

OUTPUT

Invalid input.
Transaction completed.

Q2. Hospital Patient Registration – User-Defined Exception

CODE

```
import java.util.*;
public class Main{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        try{
            String name =sc.next();
            int age = sc.nextInt();
            double temperature =sc.nextDouble();
            if(age<18||age>100){
                throw new InvalidAgeException("Invalid age");
            }
            if(temperature<95.0||temperature>104.0){
                throw new InvalidTemperatureException("Invalid temperature");
            }
            System.out.println("patient registered successfully");
        }
        catch(InvalidAgeException e){
            System.out.println("InvalidAgeException");
        }
        catch(InvalidTemperatureException e){
            System.out.println("InvalidTemperatureException");
        }
        finally{
            System.out.println("Registration completed.");
        }
    }
}
class InvalidAgeException extends Exception{
    InvalidAgeException(String message){
        super(message);
    }
}
class InvalidTemperatureException extends Exception{
    InvalidTemperatureException(String message){
        super(message);
    }
}
```

INPUT

Arun 35 98.6

OUTPUT

patient registered successfully
Registration completed.

Q3. E-Commerce Order Processing – Built-in Exceptions

CODE

```
import java.util.*;
public class Main{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        String[] products={"Laptop","Phone","Headphones"};
        double[] prices={500000,20000,3000};
        try{
            int index=sc.nextInt();
            int quantity=sc.nextInt();
            if(quantity<=0){
                throw new IllegalArgumentException("Invalid quantity");
            }
            double total=prices[index]*quantity;
            System.out.println("Total Cost = "+total);
        }
        catch(ArrayIndexOutOfBoundsException e){
            System.out.println("ArrayIndexException handled");
        }
        catch(InputMismatchException e){
            System.out.println("InputMismatchException handled");
        }
        catch(IllegalArgumentException e){
            System.out.println("Invalid quantity");
        }
    }
}
```

INPUT

abc 2

OUTPUT

InputMismatchException handled

Q4. Railway Ticket Booking – Multithreading and Synchronization

— Not Submitted —

Q5. Bank Account – Concurrent Withdrawals

— Not Submitted —

Q6. Online Food Delivery – Multiple Threads and Execution Order

CODE

```
import java.util.*;
public class Main{
    public static void main(String[] args)throws InterruptedException{
        Scanner sc=new Scanner(System.in);
        StringBuilder input = new StringBuilder();
        while (sc.hasNextLine()){
            input.append(sc.nextLine()).append(" ");
        }
        String[] data=input.toString().split(" ");
        String order=data[0].split("=")[1].trim();
        String payment=data[1].split("=")[1].trim();
        String delivery=data[2].split("=")[1].trim();
        OrderThread t1=new OrderThread(order);
        PaymentThread t2=new PaymentThread(payment);
        DeliveryThread t3=new DeliveryThread(delivery);
        t1.start();
        t1.join();
        t2.start();
        t2.join();
        if(!payment.equalsIgnoreCase("Failed")){
            t3.start();
            t3.join();
        }else{
            System.out.println("Delivery assignment should not proceed");
        }
    }
}

class OrderThread extends Thread{
    String order;
    OrderThread(String order){
        this.order=order;
    }
    public void run(){
        System.out.println("Order processing:"+order);
    }
}

class PaymentThread extends Thread{
    String payment;
    boolean success = true;
    PaymentThread(String payment){
        this.payment=payment;
    }
    public void run(){
        if(payment.equalsIgnoreCase("Failed")){
            success = false;
            System.out.println("payment processing: Failed");
        }else{
            System.out.println("payment processing: "+payment);
        }
    }
}

class DeliveryThread extends Thread{
    String delivery;
    DeliveryThread(String delivery){
        this.delivery=delivery;
    }
    public void run(){
        System.out.println("Delivery Assignment:"+delivery);
    }
}
```

Report

INPUT

Order=Burger; Payment=Card;
Delivery=Agent B

OUTPUT

Order processing:Burger
payment processing: Card
Delivery Assignment:Agent B

Q7. Hospital Emergency System – Thread Priority

CODE

```
import java.util.*;
public class Main{
    public static void main(String[] args){
        EmergencyThread emergency=new EmergencyThread();
        GeneralThread general=new GeneralThread();
        BillingThread billing=new BillingThread();
        emergency.setPriority(Thread.MAX_PRIORITY);
        general.setPriority(Thread.NORM_PRIORITY);
        billing.setPriority(Thread.MIN_PRIORITY);
        emergency.start();
        general.start();
        billing.start();
    }
}
class EmergencyThread extends Thread{
    public void run(){
        System.out.println("Emergency Patient Processing");
    }
}
class GeneralThread extends Thread{
    public void run(){
        System.out.println("General Patient Processing");
    }
}
class BillingThread extends Thread{
    public void run(){
        System.out.println("Billing Processing");
    }
}
```

OUTPUT

Emergency Patient Processing
General Patient Processing
Billing Processing

Q8. Manufacturing Plant – Producer-Consumer Problem

— Not Submitted —

Q9. Online Banking – Exception Handling with Multiple Threads

— Not Submitted —